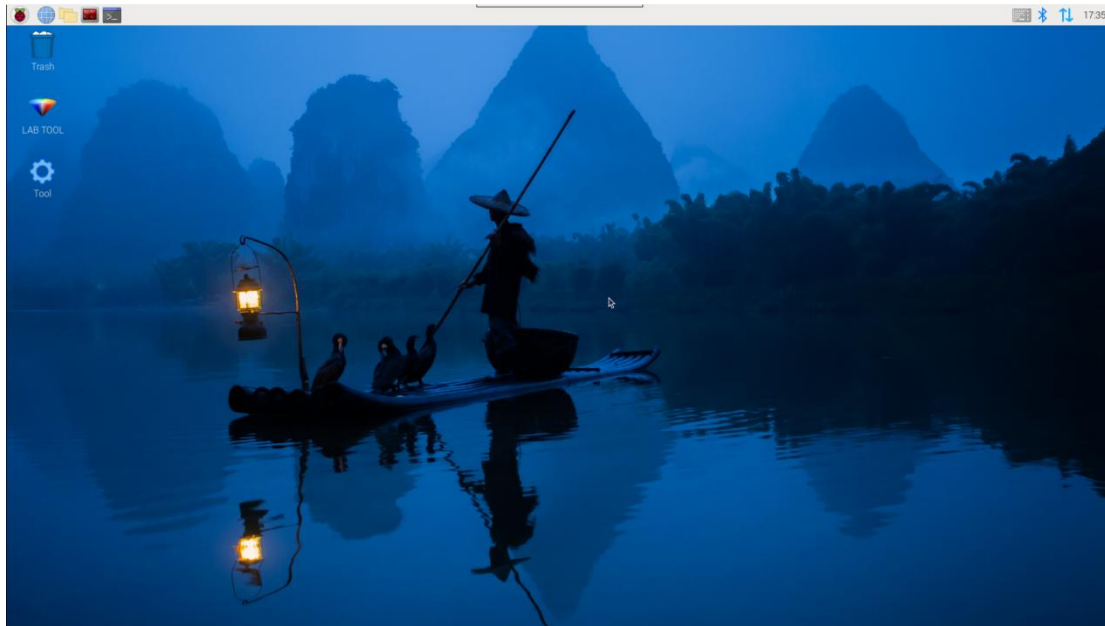
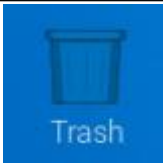


# System Introduction

## 1. Desktop Overview

After establishing a remote connection via VNC, the Raspberry Pi system desktop will appear as shown in the image below:



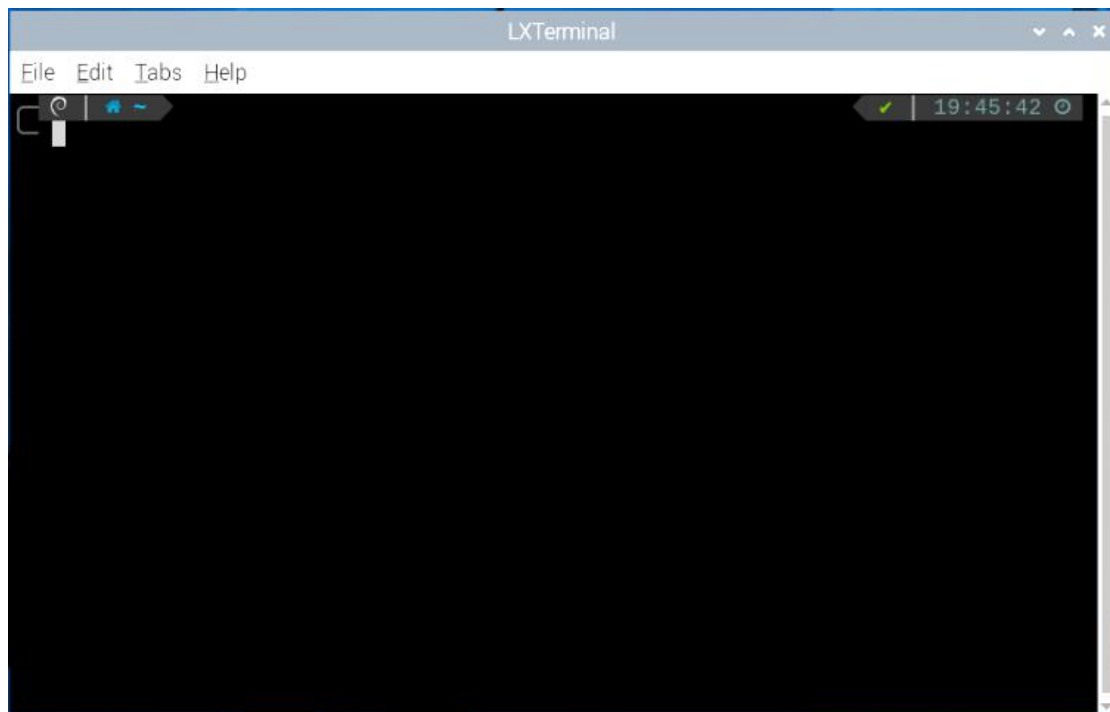
| Icon                                                                                | Function                                                                                       |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
|  | Version Switching Tool: switch between different versions and configurations of the robot car. |
|  | Color model parameter adjustment tool.                                                         |
|  | Command Line Terminal: input commands for operations.                                          |
|  | Recycle bin.                                                                                   |

|                                                                                   |                        |
|-----------------------------------------------------------------------------------|------------------------|
|  | Raspberry Pi menu bar. |
|  | System file folder.    |

## 2. Programming Instruction

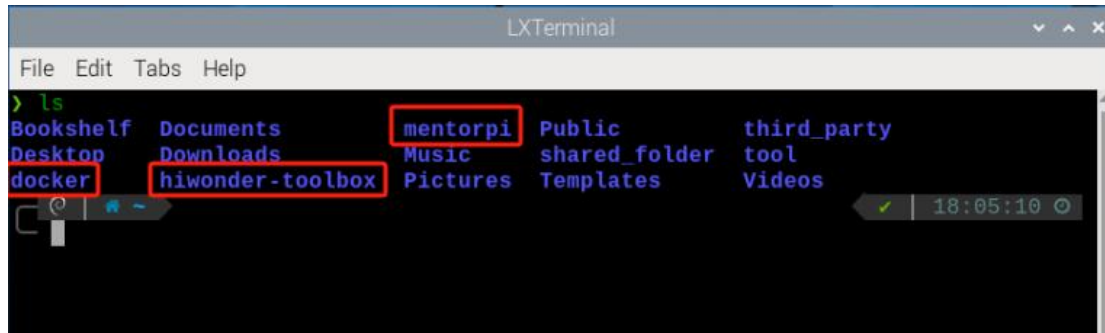
Note: commands are case-sensitive and require correct spacing. You can use the TAB key to auto-complete keywords.

- 1) Click the  or press Ctrl + Alt + T to launch the LX Terminal.



- 2) Type the following command and press Enter:

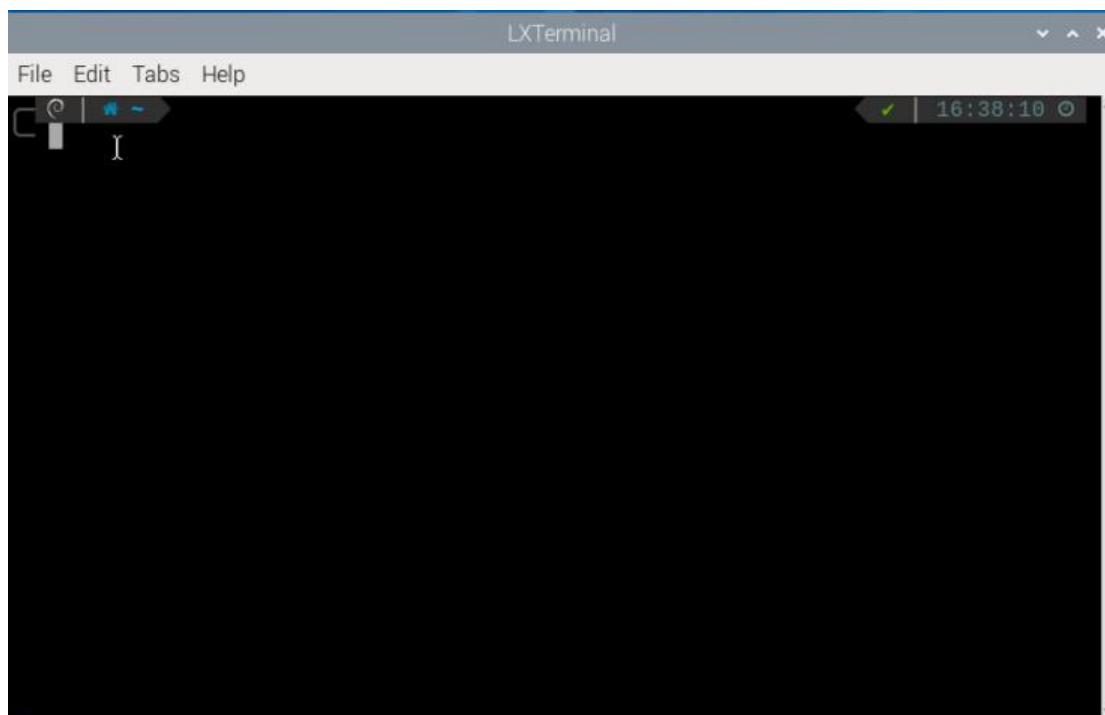
**ls**



| Directory        | Function Description                                                   |
|------------------|------------------------------------------------------------------------|
| MentorPi         | Contains debugging tools specific to MentorPi                          |
| hiwonder-toolbox | Wi-Fi management tool.                                                 |
| docker           | Shared folder between the Raspberry Pi system and the Docker container |

All function games and source code related to MentorPi are stored inside the Docker container. You must enter the container to access them.

3) Click  on the top left corner of the desktop to open a new terminal.



4) In the terminal, enter the following command to view active and previously run containers:

```
docker ps -a
```

```
ubuntu@raspberrypi: ~/ros2_ws/src
File Edit Tabs Help
> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS
f8524e126868   ros:humble "/bin/bash"             6 weeks ago   Exited (130) 6
weeks ago     objective_dhawan
adb8457c2eec   ros:humble "tail -f /dev/null"     3 months ago   Up 2 hours
```

5) Use the following command to access the container:

Note: Replace <container\_id> with the actual container ID. It can be abbreviated as long as it remains unique.

**docker exec -it -u ubuntu -w /home/ubuntu adb8 /bin/bash** (The container's ID can be abbreviated as long as it remains unique.)

```
> docker exec -it -u ubuntu -w /home/ubuntu adb8 /bin/bash
```

6) Enter the following command, and press "Enter" to list all current files. Let's focus on these two directories shown below.

ls

```
> docker exec -it -u ubuntu -w /home/ubuntu adb8 /bin/bash
ubuntu@raspberrypi:~$ ls
RRCLite_demo  ros2_ws  shared  software  third_party_ros2
```

| Directory | Function                                                        |
|-----------|-----------------------------------------------------------------|
| ros2_ws   | The ROS workspace for storing MentorPi's function game.         |
| share     | Shared directory between the Raspberry Pi and Docker container. |
| software  | Contains PC-side software and color threshold adjustment tools  |

7) Enter the command "cd ros2\_ws/src/" to access the directory of the game and source code. Then, enter the command "ls" to list all folders and files under this directory.

```
cd ros2_ws/src/
```

```
ls
```

```
ubuntu@raspberrypi:~$ cd ros2_ws/src/
ubuntu@raspberrypi:~/ros2_ws/src$ ls
app      calibration  example      navigation    simulations  yolov5_ros2
bringup  driver       interfaces    peripherals    slam
```

| Directory   | Function                        |
|-------------|---------------------------------|
| app         | Store app game function package |
| example     | Game case                       |
| bringup     | Reference to app function       |
| driver      | Underlying control              |
| interfaces  | Program interface               |
| peripherals | Hardware driver                 |
| navigation  | Navigation                      |
| slam        | Mapping                         |
| yolov5_ros2 | yolov5 game                     |
| simulations | urdf description                |